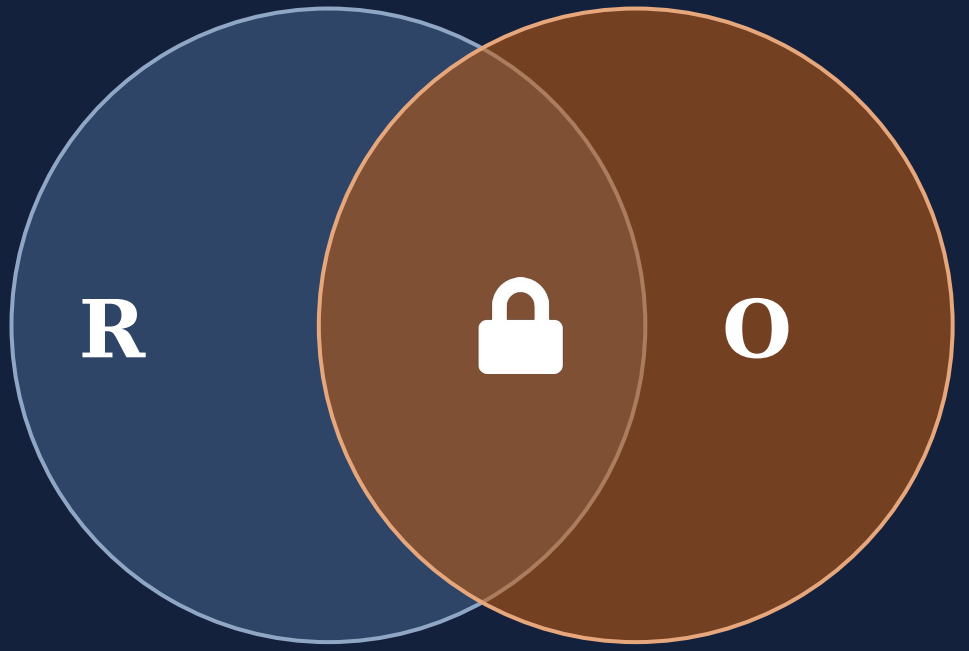
Regression across a register and a cohort, without pooling

Privacy-preserving vertical federated linear regression: motivation, protocol and results



Internal training · October 2026

What we will cover

1

The problem

Why join a research cohort to an agency register, and what must stay private

2

Building blocks

Differential privacy, homomorphic encryption, private set intersection

3

The protocol

Six phases, from setup to confidence intervals

4

Guarantees

What each party learns, and what is proven

5

Choosing parameters

From a privacy budget to noise, ridge and the sample size needed

6

Results

Accuracy in simulation and on real data, and what is still to run

1

The problem

Joint regression across data that cannot be pooled

Two datasets, one question, no pooling



Researcher R

- Smaller cohort with rich domain variables: health, education, surveys
- Wants to know whether these predict an administrative outcome
- Holds features X_R and identifiers



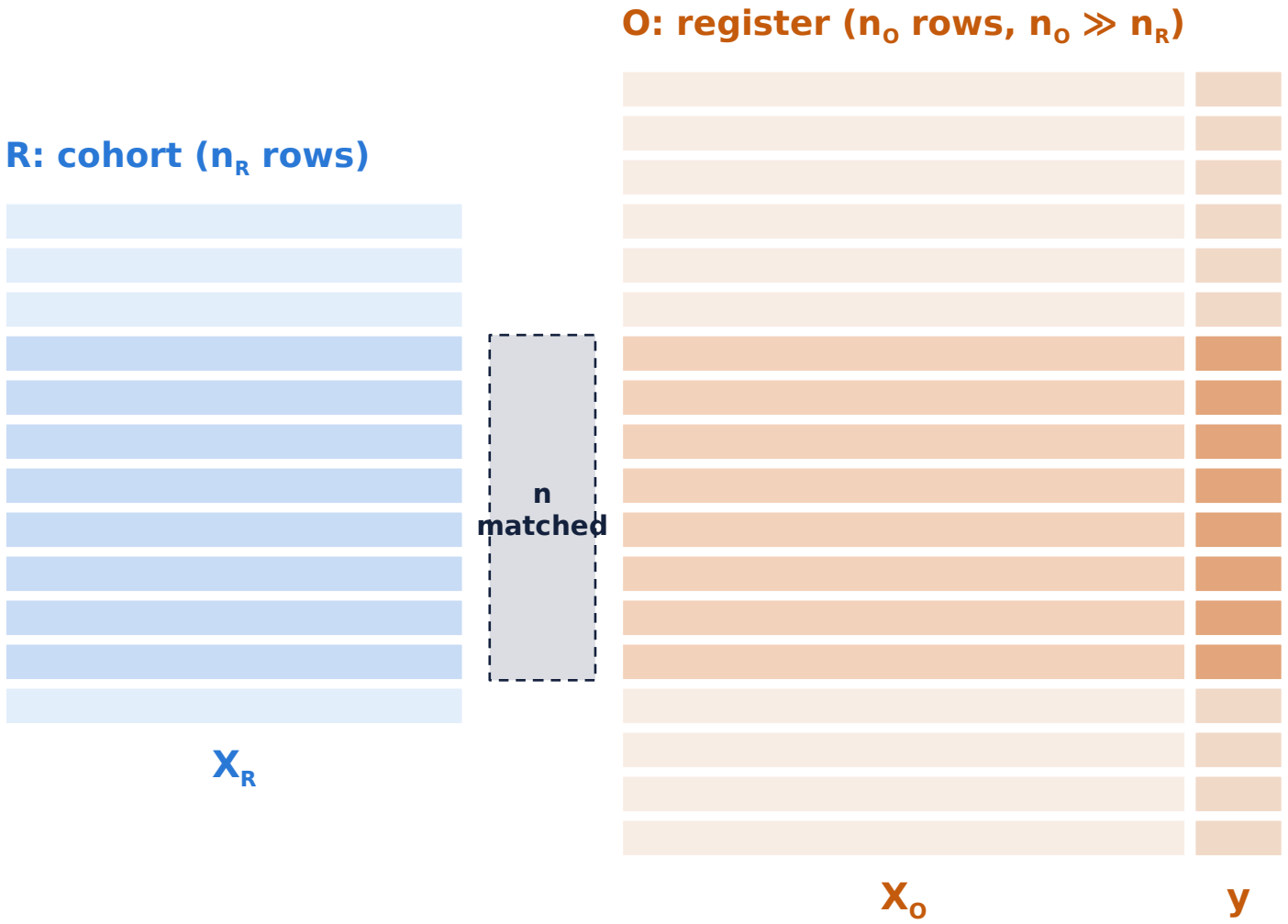
Organization O (e.g. StatCan)

- Population register: tax filings, income, benefits
- Holds the outcome y and features X_O
- Cannot release records; law and governance bar direct sharing

6–8 weeks of vetting and approvals today before a researcher can link a cohort to agency records in a Research Data Centre. Neither dataset alone answers the question.

Same people, different variables

Vertically partitioned data: overlapping rows, disjoint columns



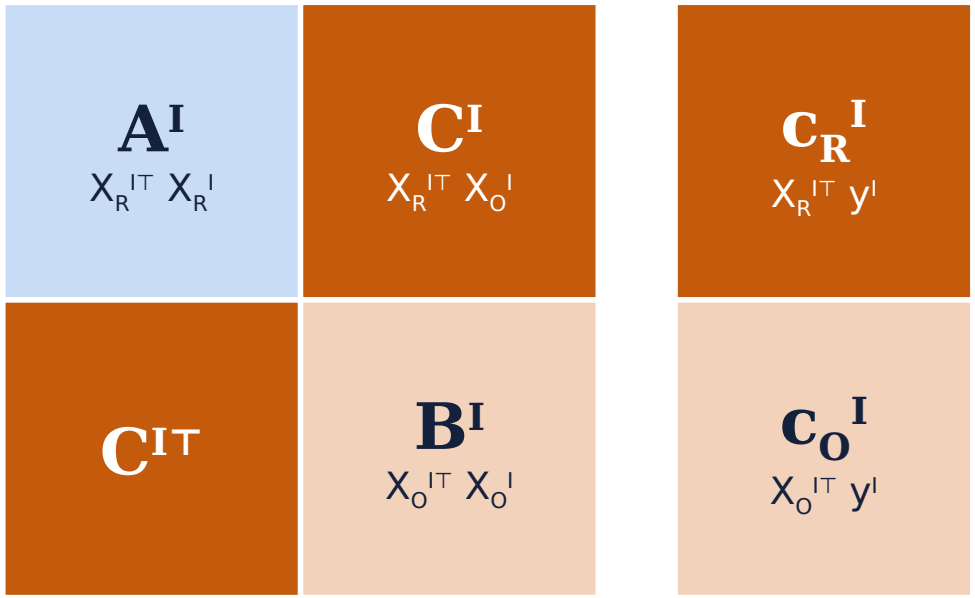
Goal

- Fit y on [X_R | X_O] over the n matched people
- Neither party sees the other's records
- O must not learn who is in R's cohort
- The released model must not expose any individual

What has to be computed

Least squares needs only a few sums over the matched rows, and those sums span both parties

$$\hat{\beta} = (Z^\top Z)^{-1} Z^\top y^I, \qquad Z = \left[X_R^I \mid X_O^I \right]$$



$G^I = Z^\top Z$ $\qquad$ $c^I = Z^\top y^I$

R's rows in I only: R computes it exactly, and it never leaves R

O's rows π(I) only, as is $(y^\top y)^I$: depends on which rows match, not on how they pair

Cross-party: pairs R's row i with O's row π(i), so it depends on the bijection π

I = the cohort members found in O's register. **π** = the bijection from I to O's matching rows; X_O^I and y^I list O's rows π(i) in the order of I. Everything that touches O's data is computed under encryption and released to R only with noise. From here on we drop the superscript I.

Three requirements

Input privacy

O learns nothing about R's features. Its whole view can be simulated from public sizes.

Cohort privacy

O learns nothing about who is in R's cohort, not even its exact size.

Output privacy

Everything R receives is differentially private for each person's O-side data (x_O, y).

Honest-but-curious parties. Both follow the protocol but analyse everything they receive. R and O each hold a share of the decryption key. By design, R learns which members of its cohort are in the register.

Where this work sits

Approach	Where the data sit	Record linkage	Output privacy	Gap for this setting
Central DP regression (e.g. AdaSSP)	One trusted curator holds the joined table	Not needed	Differential privacy	Assumes the pooled data exist
Federated learning with encryption	Split across parties	Assumed, or a separate step	Usually none	The model can leak individuals
Private set intersection	Split across parties	Yes	Not applicable	Linkage only, no model
This protocol	Split, never pooled	Labeled PSI: O learns nothing	DP on all R receives; budget enforced by O	Protects O's side; R learns which of its cohort match

The statistical engine is the classic one for DP regression: perturb the sufficient statistics, then solve a ridge regression. What is new is doing it without pooling, with R's own block kept exact.

2

Building blocks

Differential privacy, homomorphic encryption, private set intersection

Differential privacy in one slide

A release is private if no single person's data changes it by much

$$\Pr[\mathcal{M}(D) \in S] \;\leq\; e^{\varepsilon}\,\Pr[\mathcal{M}(D') \in S] + \delta$$

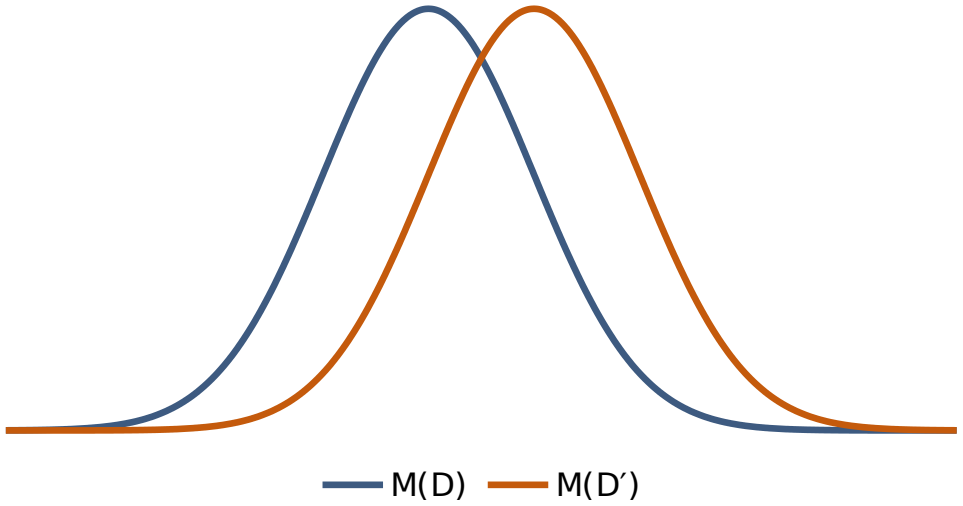
- D and D′ differ in one person's O-side data (x_o, y): replace-one adjacency
- ε caps how much one person can shift the odds of any outcome (a factor e^{ε})
- δ is a small slack; take δ ≤ 1/(10N) for a register of N people
- Post-processing is free: anything computed from a DP release stays DP

The Gaussian mechanism

$$\mathcal{M}(D) = f(D) + \mathcal{N}(0, \sigma^2 I), \qquad \sigma = \Delta \cdot s(\varepsilon, \delta)$$

$$\Delta = \max_{D \sim D'} \|f(D) - f(D')\|_2$$

Outputs on neighbouring datasets



The two curves are shifted by the sensitivity Δ. The wider the noise relative to Δ, the harder it is to tell them apart.

How much one person can move the statistics

The sensitivity Δ sets the noise; it follows from public clipping bounds

Every row is clipped to public bounds: $\|x_R\| \le B_R$ (with the intercept), $\|x_O\| \le B_O$ and $|y| \le B_y$. Replacing one person's (x_O, y) changes B, C, c_R, c_O and y^Ty by at most Δ:

$$\Delta^2 = \tfrac{1}{2}(2O + R + Y)^2 + 2RY, \qquad (R, O, Y) = (B_R^2,\ B_O^2,\ B_y^2)$$

Bounds (B_R, B_O, B_y)	Δ	Comment
(1, 1, 1)	3.16	unit bounds
(0.71, 0.71, 1)	2.03	one unit row bound split between the parties
(5.59, 6.24, 2.89)	86.2	simulation population (standardized income, age, counts)

Bounds are chosen from public reference data before any data are seen; values beyond them are clipped.

- $A = X_R^T X_R$ is computed exactly by R and never released, so it adds nothing to Δ
- y^Ty is free when $B_y^2 \le 2(B_R^2 + B_O^2)$; otherwise it gets its own budget
- Δ grows with the squared bounds: tight bounds matter most

Budgets add up, and O keeps the books

Zero-concentrated DP (zCDP) makes the accounting simple

$$\rho_s = \frac{\Delta_s^2}{2\sigma_s^2}, \qquad \rho_{\text{run}} = \rho_1 + \rho_2$$

$$\sqrt{\rho_{\text{run}}} \gtrsim \sqrt{L+\varepsilon} - \sqrt{L}, \qquad L = \ln(1/\delta)$$

- Each release s costs ρ_s; the costs of a run add up
- The run's total converts to (ε, δ); the protocol uses the tight conversion, and the formula above is a close, simple bound
- Doubling ε roughly halves the noise σ

One run = two releases



Stage 1: B and C only, 10% of ρ, used to choose the ridge

Ledger at O

One ledger per register, shared by all researchers. O refuses a run once the cap would be passed. Nothing can be decrypted without O's share, so the cap is enforced technically.

Homomorphic encryption

Computing on ciphertexts. CKKS: approximate arithmetic on encrypted vectors of real numbers

- A ciphertext holds a whole vector: N/2 slots ($N = 2^{14}$ gives 8,192)
- O can add ciphertexts, multiply them by its own plaintext numbers, and rotate slots to sum them
- Values are scaled by Ω and rounded; decryption returns them plus a small error e
- Security rests on the Ring-LWE problem

Decryption with secret key s

$$c_0 + c_1 s \;=\; \mathrm{Ecd}(z) + e \;\pmod{q}$$

A ciphertext is a pair (c_0, c_1) of large polynomials.



Split key (threshold decryption)

The secret key is the sum of two shares, one per party. Decrypting needs both:

$$s = s_R + s_O: \qquad c_0 + c_1 s \;=\; \underbrace{(c_0 + c_1 s_O)}_{\text{needs O}} + \underbrace{c_1 s_R}_{\text{needs R}}$$

So neither party can decrypt alone, and O can attach its noise to its own part of every decryption.

Private set intersection

Linking without revealing. Labeled PSI (Chen, Laine and Rindal): R learns its matches, O learns nothing

1 Blind the identifiers

R gets a keyed hash $F_k(\text{id})$ of each cohort id from O (an OPRF), without O seeing the ids. R pads to a public cap n_R^{max}.

2 Match under encryption

R sends encrypted queries; O evaluates polynomials built from its own hashed register and returns encrypted answers.

3 Read the result

R decrypts: which ids match, and a label for each match, here the row's position in O's secretly shuffled register.

R learns

The match set I and each match's position in O's shuffled order: enough to line its data up with O's rows later.

O learns

Only the padded sizes n_R^{max} and $\bar{n}_O$. Its shuffle is fresh for every run, so positions reveal nothing.

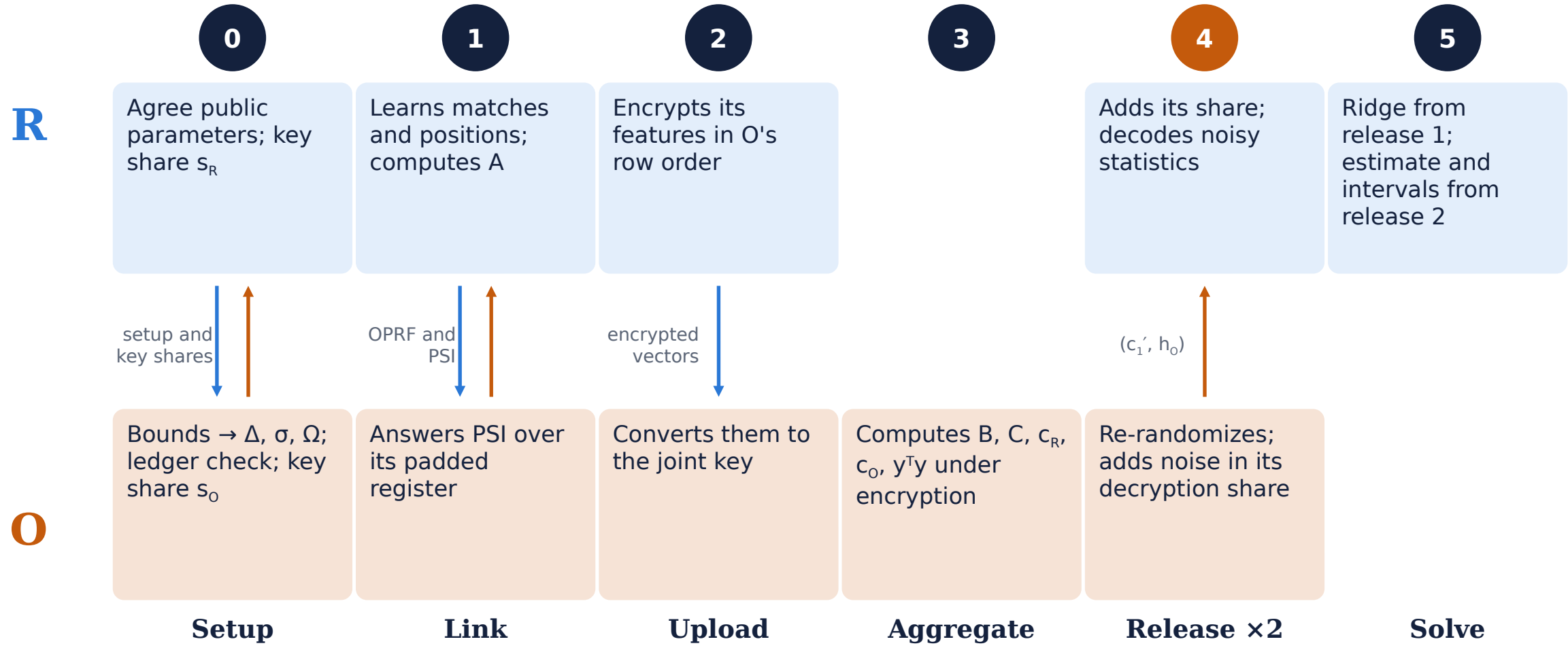
3

The protocol

Six phases, from setup to confidence intervals

The protocol at a glance

What each party does, and what crosses the wire



O never receives a decryption share from R; R never receives the part of a ciphertext that would expose O's key share.

Phase 0: setup

Everything that sets the noise is public and fixed before any data are touched

R + O Agreed in public

- Study scope Φ, on public attributes only (province, birth-year band)
- Caps: cohort n_R^{max}; register padded to $\bar{n}_O$ with dummy rows
- Clipping bounds B_R (with the intercept), B_O, B_y, from public reference data
- Budget (ε, δ); stage-1 share $f_1 = 0.1$; validity target $\vartheta^* = 2.5$; error level α

O Derived by O

- Sensitivities Δ and Δ_G from the bounds
- Noise σ_1, σ_2 and the CKKS scale Ω (Calibrate)
- Ledger check: refuse if the run would pass the cap
- A fresh secret shuffle τ of its rows and a fresh OPRF key

R + O Split key generation

Joint public key, secret $s = s_R + s_O$, and rotation keys only. Neither party can decrypt alone.

Phase 1: link

Labeled PSI tells R which cohort members are in the register, and where

R Pad the cohort identifiers with random dummies to exactly n_R^{max}

R + O OPRF: R gets $F_k(id)$ for each padded id; O evaluates exactly n_R^{max} calls

R Cuckoo-hash the hashed ids; send encrypted queries under R's own key

O Evaluate membership and label polynomials over its padded register; reply

R Decrypt: match set I, and positions $\pi(i) = \tau(j)$ in O's shuffled order

R Compute its own block $A = \Sigma_{i \in I} \dot{x}_i \dot{x}_i^T$ exactly, locally

After phase 1

- **R knows I and the positions τ(j) of its matches**
- R has A exactly; it never leaves R
- **O knows only n_R^{max} and $\bar{n}_O$**
- The positions are uniformly random, because τ is fresh every run

Phase 2: upload

R lines its data up with O's rows, encrypts them, and sends them over

R

Vectors over all $\bar{n}_o$ positions

- Selection b: 1 at a matched position, 0 elsewhere
- Features $\dot{x}_j$: R's clipped feature j (and the intercept) at matched positions, 0 elsewhere
- In O's shuffled order, so O can multiply them by its own columns position by position

Why this hides everything from O

O sees only ciphertexts, which look random without R's key share. Matched and unmatched positions are indistinguishable, so O cannot tell who is in the cohort.

R : $c_0 = -a\, s_R + e + \mathrm{Ecd}(v), \quad a = \mathrm{PRG}(\mathrm{seed})$ O : $c_0 \leftarrow c_0 - a\, s_O$

R encrypts with its own share and a short seed (half the upload); O folds in its share to convert to the joint key.

Phase 3: aggregate

O multiplies R's encrypted vectors by its own plaintext columns and sums

$$\mathrm{Enc}(B_{jk}) = \sum_i \mathrm{Enc}(b_i)\, X_{O,ij}\, X_{O,ik}$$

$$\mathrm{Enc}(C_{jk}) = \sum_i \mathrm{Enc}(\dot{X}_{R,ij})\, X_{O,ik}$$

$$\mathrm{Enc}(c_{O,k}) = \sum_i \mathrm{Enc}(b_i)\, X_{O,ik}\, y_i$$

$$\mathrm{Enc}(c_{R,j}) = \sum_i \mathrm{Enc}(\dot{X}_{R,ij})\, y_i$$

b: selection vector; $\dot{x}_R$: R's features in O's order (zero off the match set)

Sum chunks, then EvalSum

A register longer than one ciphertext is split into chunks; slot sums use rotations.

Pack: one entry per slot

Every released number sits alone in its own slot of a packed ciphertext.

Two packed ciphertexts

M_1 holds only B and C (for stage 1); M_2 holds every entry (stage 2).

Unmatched positions contribute zero, because b and $\dot{x}_R$ are zero there. The result is the matched-row statistics, still encrypted. Multiplicative depth is 2.

Phase 4: the release

O adds the privacy noise inside its own share of the decryption

$$
\begin{aligned}
\text{O}: \quad & c_0' = c_0 + p_0 u + e_0, \qquad c_1' = c_1 + a u + e_1 \\
& h_O = c_0' + c_1' s_O + \eta, \qquad \eta \sim \mathcal{N}_{\mathbb{Z}}(0, \sigma_c^2)^N \\
& \text{send } (c_1', h_O) \\
\text{R}: \quad & v = h_O + c_1' s_R = \mathrm{Ecd}(z) + e + \eta
\end{aligned}
$$

c_0' never leaves O

Without it R cannot work out $c_1's_O$, so O's key share stays hidden.

η does two jobs

It is the DP noise on the statistics, and it hides every encryption error statistically (to 2^{-40}).

Fresh every time

New u, e_0, e_1 and η for each release; O charges the ledger before sending.

What R ends up with: the statistics plus discrete-Gaussian noise of the planned size, and a c_1' that looks random. R's view can be simulated from the noisy statistics alone, which is exactly the DP guarantee.

Why the noise must cover the whole ciphertext

100%

of trials in which R picked the true dataset out of two candidates (20 of 20, real OpenFHE), when O added the noise only to the plaintext value.

73%

the most that ε = 1 differential privacy allows.

What went wrong

- Adding noise to the plaintext changes only c_0
- c_1 stays a deterministic function of R's ciphertexts and O's data
- Holding a key, R can recompute c_1 for each candidate dataset and compare

How we avoid it

- O re-randomizes c_1 before releasing, so c_1' looks random
- The noise sits in O's decryption share, so it covers everything R can compute
- The error terms are hidden statistically by the same noise

Lesson: DP must hold for everything a party sees, not only for the decrypted number.

Two releases: pick the ridge, then estimate

The ridge must not depend on the noise in the release it is applied to

Stage 1 (10%)

Release of B and C only, noise σ_1

→

$\hat{G}_1$ = A ⊕ noisy B, C

R adds its exact block A; finds a lower bound on $\lambda_{min}(G)$

→

Choose (λ, Ψ)

Ridge just large enough that ϑ ≥ 2.5; Ψ ridges only O's block if that suffices

→

Stage 2 (90%)

All statistics, noise σ_2; gate $\hat{\vartheta} \geq 1$

$$\lambda = \max\{0,\ 2\vartheta^\star \sigma_2 \sqrt{p} + r_1 - \lambda_{\min}(\hat{G}_1)\}, \qquad r_1 = \sigma_1\left(2\sqrt{p} + 2\sqrt{\ln(1/\alpha_L)}\right)$$

$$\tilde{\theta} = (\hat{G}_2 + \lambda\Psi)^{-1}\hat{c}_2, \qquad \hat{\vartheta} = \frac{\lambda_{\min}(\hat{G}_2 + \lambda\Psi)}{2\sigma_2\sqrt{p}} \ \geq\ 1$$

Because (λ, Ψ) comes from stage 1 and A only, it is independent of the stage-2 noise. That keeps the error analysis and the intervals valid. ϑ measures how far the noisy Gram matrix is from singular.

Phase 5: estimates and intervals

Everything here is post-processing at R: it costs no extra privacy budget

Two targets

$$\theta_\lambda = (G + \lambda\Psi)^{-1}c \qquad \text{and} \qquad \beta = G^{-1}c$$

θ_λ is what the protocol estimates directly (the ridge target on the clipped data); β is the least-squares coefficient.

Parametric bootstrap of the noise

- R knows the noise distribution exactly, so it simulates many fresh noise draws around its release
- This gives the mechanism's covariance V_{mech}, and corrects the residual sum of squares for the noise
- Add the usual sampling variance to get the total V

Interval for θ_λ (Wald)

$$\tilde{\theta}_j \;\pm\; z_{1-\alpha/2}\sqrt{(V_{\mathrm{mech}})_{jj}}$$

Covers the protocol's own target at the nominal level.

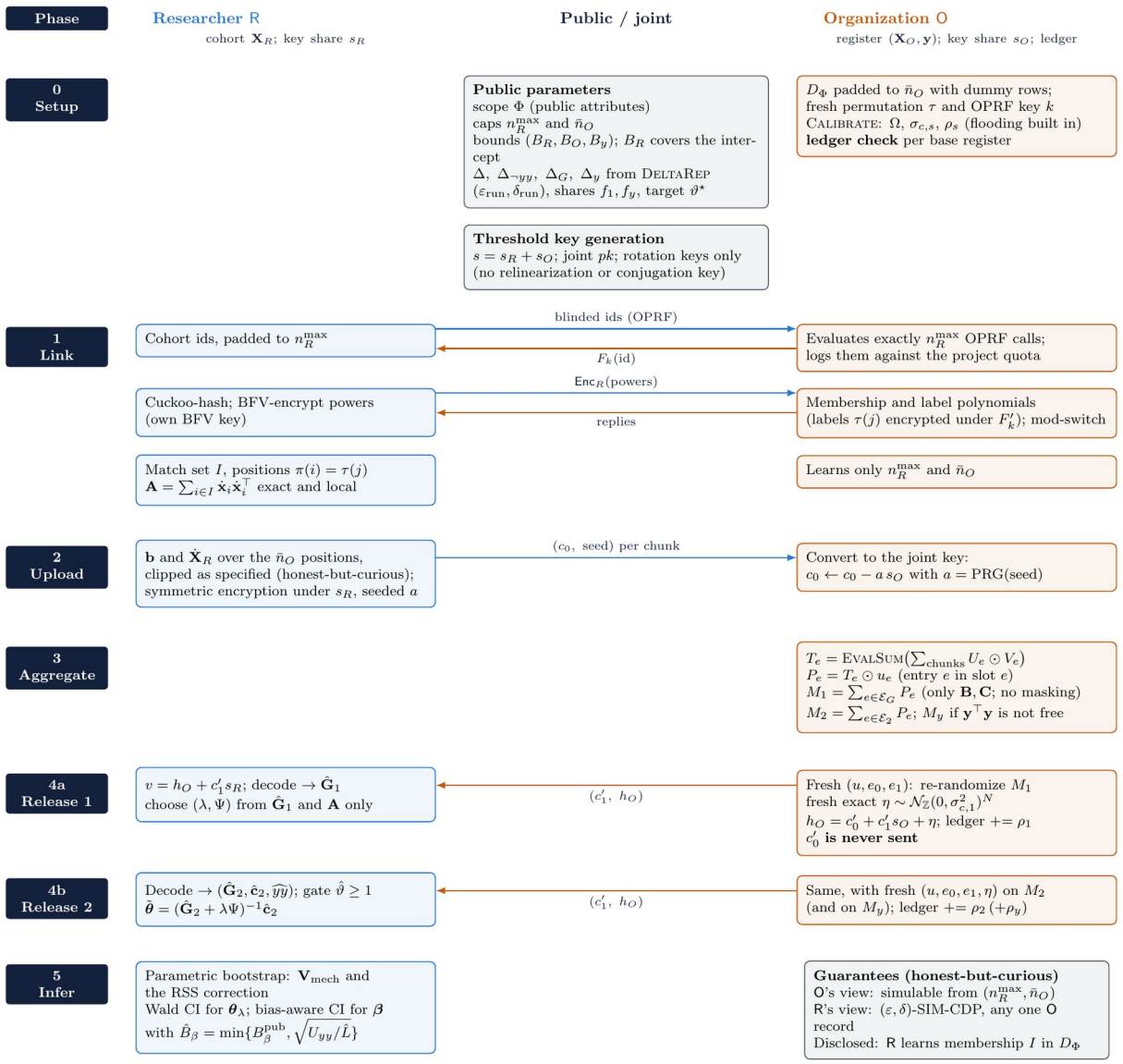
Interval for β (bias-aware)

$$\tilde{\theta}_j \;\pm\; \mathrm{cv}_\alpha\big(\bar{b}_j/\sqrt{V_{jj}}\big)\sqrt{V_{jj}}, \qquad \bar{b}_j = \lambda\,\|\Psi\tilde{P}e_j\|\,\hat{B}_\beta$$

Widened for the worst-case ridge bias (Armstrong, Kolesár and Kwon); needs a bound on $\|\beta\|$.

The whole protocol on one page

Blue arrows: R to O. Orange arrows: O to R.



4

Guarantees

What each party learns, and what is proven

What each party learns

Against honest-but-curious parties

	Learns	Does not learn	Why
O	The public sizes n_R^{max} and $\bar{n}_O$	R's features, who is in the cohort, or which of its rows match	Ring-LWE in R's key share; PSI security; O never receives a decryption share from R
R	Its own data; which of its cohort match; the DP-noised statistics	Anything about one person's O-side data beyond (ε, δ)-DP	Noise in O's decryption share; the release can be simulated from the noisy statistics (SIM-CDP)

Assumed

- Both parties follow the protocol (honest-but-curious)
- Ring-LWE; security of the PSI and of threshold key generation
- Private, authenticated channels

Not covered

- A researcher who deviates, e.g. skips clipping: needs zero-knowledge input proofs
- Hiding match status from R: possible with circuit-PSI, at a higher sensitivity

What is proven

A 47-page proofs appendix: cryptography and DP (part A), sensitivity and statistics (part B)

Cryptography and DP	Status
Sensitivity of the encoded statistic	**Proven**
Noise hides the encryption error (smudging), explicit constants	**Proven**
Worst-case encryption error $B_{tot} \approx 2^{25.3}$; OpenFHE measures 6–8 bits below it	**Proven**
zCDP accounting; $\rho_{run} = 0.0150$ at $\varepsilon = 1$, $\delta = 10^{-9}$	**Proven**
Output privacy (SIM-CDP), by a sequence of hybrids	**Proven**
Input and cohort privacy; correctness without wraparound	**Proven**

Sensitivity and statistics	Status
Replace-one sensitivity, exact for $d_O \geq 2$	**Proven**
When $y^T y$ is free	**Proven**
Stage-1 ridge choice independent of the stage-2 noise	**Proven**
Lower bound on $\lambda_{min}(G)$ holds with probability $\geq 1 - \alpha_L$	**Proven**
Interval coverage (simulated: 0.944–0.962 against 0.955)	**First order**
Bootstrap correction of the residual sum of squares	**First order**

Still open: proofs for the improved circuit; composition across adaptive runs; a constant-time exact noise sampler; exact modulus sizes; heteroskedasticity when clipping binds.

Why the CKKS scale has to be large

Hiding the encryption error statistically costs about 40 extra bits

$$\mathrm{SD}(\eta + e,\ \eta) \ \le\ \frac{\|e\|_2}{\sqrt{2\pi}\,\sigma_c} \ \le\ 2^{-\kappa} \quad \text{if} \quad \sigma_c \ge \frac{2^{\kappa} B_{\mathrm{tot}}}{\sqrt{2\pi}}$$

$$\sigma_2 = \sigma_c \sqrt{N/2}\,/\,\Omega$$

The privacy budget fixes the noise σ_2 seen in the statistics. So the scale Ω must be large enough that η is also 2^{40} times the error:

$$\log_2 \Omega \ \approx\ \underbrace{\kappa}_{40} + \underbrace{\log_2 B_{\mathrm{tot}}}_{25.3} + \underbrace{\log_2 \sqrt{N/2}}_{6.5} - \underbrace{\log_2 \sqrt{2\pi}}_{1.3} - \underbrace{\log_2 \sigma_2}_{9.0} \ \approx\ 61.5$$

Example: $\varepsilon = 1$, $\delta = 10^{-9}$, $N = 2^{14}$, simulation bounds ($\sigma_2 = 525$).

Larger scales mean larger moduli; exact parameter sets are still to be fixed.

Circuit	B_{tot}	$\log_2 \Omega$
As specified	$2^{25.3}$	61.5
Sum before the first rescale; stochastic rounding; one final rescale	$2^{21.1}$	57.2
Same, with R encoding at scale 2^{60}	$2^{17.6}$	53.7

$2^{14.0}$–$2^{15.7}$

measured error in OpenFHE for the same circuit shape, against a worst-case bound of $2^{21.9}$. O must calibrate from the worst-case bound, never from the observed error, which depends on the data.

5

Choosing parameters

From a privacy budget to noise, ridge and the sample size needed

From ε to σ and λ

Every input is public and fixed before any data are seen

1 δ

$\delta \le 1/(10N)$

N = people O protects. Smaller δ costs little (next slide)

2 Budget

tight conversion of (ε, δ)

$\sqrt{\rho_{\text{run}}} \gtrsim \sqrt{L+\varepsilon} - \sqrt{L}$

L = ln(1/δ). Two releases share ρ_{run}

3 Δ

$\Delta^2 = \frac{1}{2}(2O + R + Y)^2 + 2RY$

(R, O, Y) = squared bounds; Δ_G: B and C only

4 σ

$\sigma_1 = \Delta_G / \sqrt{2 f_1 \rho_{\text{run}}}$

$\sigma_2 = \Delta / \sqrt{2(1-f_1)\rho_{\text{run}}}$

$f_1 = 0.1$ is the stage-1 share

5 λ

$\lambda \approx \max\{0,\ 2\vartheta^\star \sigma_2 \sqrt{p} + r_1 - n\ell\}$

Planning value; $\ell = \lambda_{\min}(G)/n$ from reference data, else 0

σ_2 / Δ δ \ ε	0.1	0.25	0.5	1	2	4	8
$\delta = 10^{-6}$	41.6	17.6	9.15	4.78	2.51	1.34	0.727
$\delta = 10^{-8}$	51.6	21.4	11	5.68	2.95	1.55	0.825
$\delta = 10^{-9}$	56	23.1	11.9	6.09	3.15	1.64	0.871

Planning formula

$$\sigma_2 \le \frac{\Delta}{\sqrt{2(1-f_1)}\left(\sqrt{L+\varepsilon} - \sqrt{L}\right)}$$

Uses the simple zCDP bound, so it overstates σ_2 by 6–33%. O computes the exact σ for each release.

From σ to the matched records you need

Two scaling laws, exact at first order: one pilot figure carries over to any ε

Prediction

$$n \propto \sigma \propto \Delta/\varepsilon$$

The ridge scales with σ, so the excess test error depends on n only through n / σ. Doubling ε roughly halves the records.

Coefficients

$$n \propto \sigma^2 \propto (\Delta/\varepsilon)^2$$

Once the ridge is off, DP error / sampling error ≈ $\sigma^2(1+\|\beta\|^2)\ \mathrm{tr}(\Sigma^{-2}) / (n\, s^2\, \mathrm{tr}(\Sigma^{-1}))$. Doubling ε cuts the records by about 3.5×.

Target	Setting	First-order calculator	Transfer (× σ or × σ²)	Observed
Prediction within 10%	bike, ε = 2 (from ε = 1)	5.7k	5.8k	**5.9k**
Prediction within 10%	pol, ε = 4 (from ε = 1)	2.8k	2.9k	**2.8k**
Prediction within 10%	protein, ε = 4 (from ε = 1)	747	794	**619**
Coefficients within 2× OLS	R^2 = 0.5, ε = 4 (from ε = 2)	11k	10.0k	**9.4k**
Coefficients within 2× OLS	R^2 = 0.5, ε = 8 (from ε = 2)	3.3k	3.1k	**3.2k**
Coefficients within 2× OLS	R^2 = 0.3, ε = 1 (from ε = 2)	78k	74k	**75k**

Checked on the experiments of part 6 (one release; prediction on real data, coefficients in simulation). The laws carry over to the protocol's σ_2. They break down for ill-conditioned designs, at high R^2 where clipping bias dominates, and at small n.

What each choice costs

Defaults: $\delta = 10^{-9}$, $f_1 = 0.1$, $\vartheta^* = 2.5$, tight bounds, one run per budget

Change	σ_2	Records for prediction	Records for coefficients
Bounds 2× closer to the data (no extra clipping)	× 0.25	× 0.25	× 0.06
Split one unit row bound between the parties	× 0.64	× 0.64	× 0.41
Double ε	× 0.51–0.53	× 0.51–0.53	× 0.26–0.28
δ from 10^{-6} to 10^{-9}	× 1.20–1.32	× 1.20–1.32	× 1.44–1.73
Stage-1 share 0.2 instead of 0.1	× 1.06	× 1.06	× 1.12
One budget shared by 5 runs (zCDP)	× 2.24	× 2.24	× 5.00
One budget shared by 5 runs (naive split of ε, δ)	× 4.81–4.97	× 4.81–4.97	× 23.10–24.70

Worked example: the simulation population, $\delta = 10^{-9}$ ($\Delta = 86.2$, $\Delta_G = 77.2$, $p = 11$, $\ell = 0.494$)

ε	ρ_{run}	σ_1	σ_2	λ at n = 10k	Ridge off from n	…if ℓ were known exactly
0.5	0.0040	2744	1021	43633	**98k**	31k
1	0.0150	1410	525	20018	**51k**	16k
2	0.0561	728	271	7951	**26k**	8.2k
4	0.2063	380	141	1784	**14k**	4.3k
8	0.7328	202	75	0	**7.2k**	2.3k

6

Results

Accuracy in simulation and on real data, and what is still to run

How we evaluated

Simulation population

Features on real-world scales (income, age, binary indicators, counts), correlated. Public standardization constants and 99.5% bounds. $R^2 \in \{0.3, 0.5, 0.9\}$; n from 100 to 100k; 200 replicates.

Real data

29 UCI regression datasets from Wang's AdaSSP study, with his preprocessing; 10-fold cross-validation; $\delta = 10^{-6}$.

Comparisons

Non-private least squares (the floor); predicting zero (the trivial baseline); AdaSSP, central DP on pooled data.

What the experiments simulate

- One DP release of the statistics (analytic-Gaussian σ), then the ridge solve. The protocol's two releases give a σ_2 that is 1.11–1.16× larger at the same (ε, δ).
- The ridge was set from a known eigenvalue bound in simulation, and from $\ell = 0$ on real data. The protocol learns it from stage 1 instead (previous slide).
- The optional bias correction was applied; on real data it changes test MSE by −3.7% to +2.3%. Reruns under the full protocol are next.

Linkage and aggregation are exact

A prototype of these two steps, run end to end over sockets (single key, no release step)

0

misaligned rows

PSI recovered exactly the true match set in all 6 test cohorts, plaintext and OpenFHE

$\approx 10^{-10}$

max |β − plaintext answer|

noise switched off: encryption adds no error to the regression

≈13 s

end-to-end on OpenFHE

$n = 400$ matched out of $n_O = 4{,}000$ records (OpenFHE 1.5.1)

Cost model of the full protocol, per run

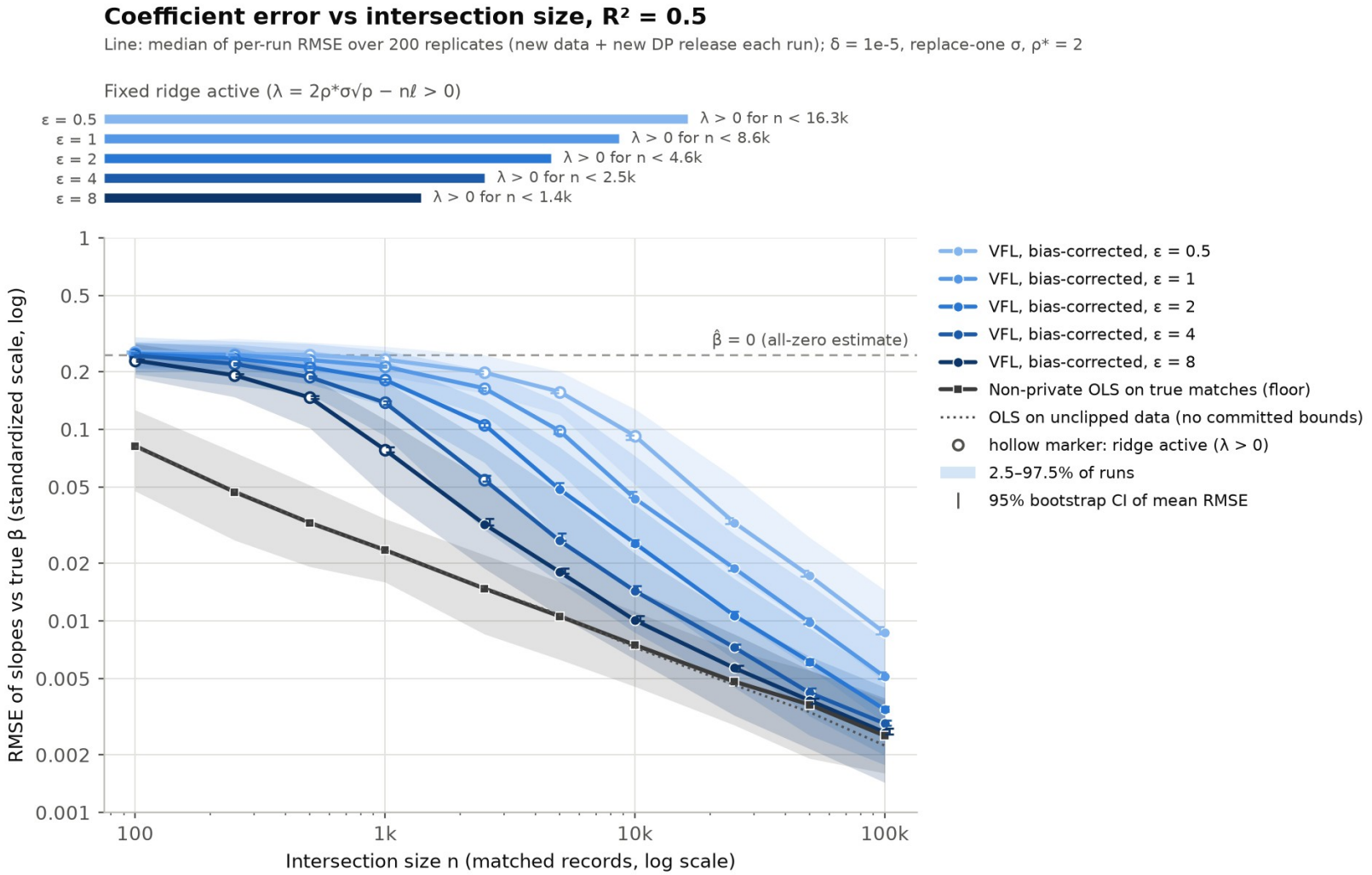
Benchmarks to be run

Step	Communication	O's computation	Rounds
Link	n_R^{max} OPRF calls; PSI query and reply	Preprocessing linear in $\bar{n}_O$; per-query evaluation	2–3
Upload	$(d_R + 2)\cdot m$ half-size ciphertexts	$(d_R + 2)\cdot m$ key conversions	1
Aggregate	None	$\lvert\mathcal{E}\rvert\cdot m$ ciphertext–plaintext products; $\lvert\mathcal{E}\rvert\cdot \log_2(N/2)$ rotations	0
Release (×2)	Two ring elements each	Re-randomization, partial decryption, N exact noise samples	1 each

$m = \lceil \bar{n}_O/(N/2) \rceil$ chunks; $|\mathcal{E}|$ = number of released entries. O's work grows with the padded register $\bar{n}_O$, so the public scope Φ is the main lever on cost.

Accuracy against intersection size

Simulation, $R^2 = 0.5$, one release per run

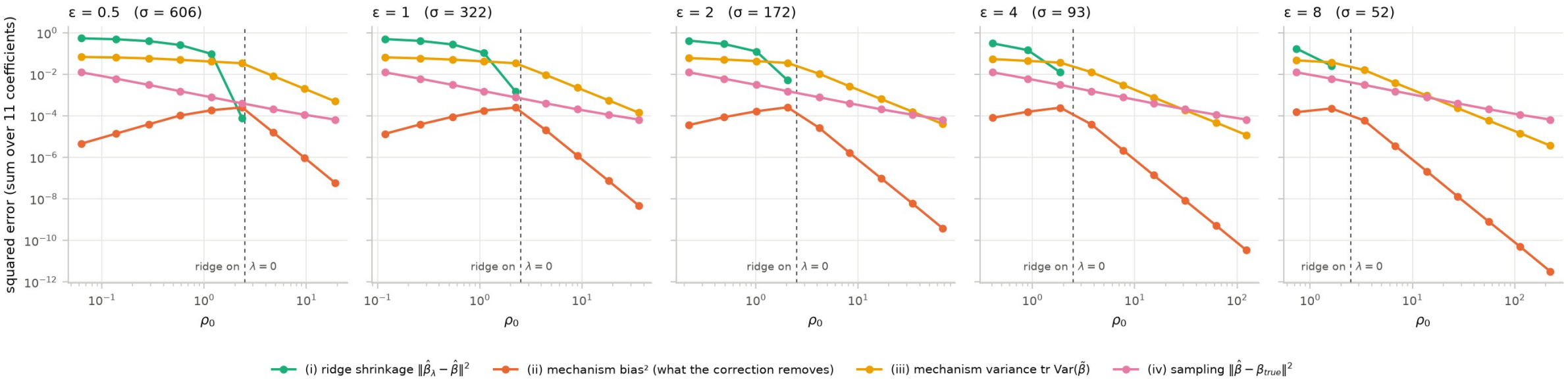


Within 2× of non-private least squares at

ε = 8	**n ≈ 3.2k**
ε = 4	**n ≈ 9.4k**
ε = 2	**n ≈ 34k**
ε = 1	**n ≈ 105k**

- Small n: the ridge pulls the estimate towards 0
- Then the DP error falls like 1/n and merges into the sampling floor
- Run-to-run spread is about ×2.5–3

Where the error comes from when σ is large



Error anatomy

- Heavy ridge: 72–90% of the error is shrinkage, 11–31% noise variance
- Bias from the noise itself: at most 0.75% of the error
- At large n, clipping shifts the target slightly (any DP method pays this)

So what?

- The ridge, not the noise, limits accuracy at small n: tighter bounds and more matched records help most
- A second-order bias correction exists but changes little, so it is optional and off by default

Intervals that hold up

Simulation of the release and the solve step, 2,000 replications per scenario

Scenario	n	Ridge	Wald interval for θ_λ (target 0.955)	Bias-aware interval for β (target ≥ 0.95)
Ridge off	4,000	$\lambda = 0$	0.946–0.958	0.945–0.968
Ridge on, whole matrix	1,500	$\Psi = I$, median $\lambda = 477$	0.948–0.960	1.000 with a public bound on $\|\beta\|$; unbounded without
Ridge on, O's block only	3,000	$\Psi = \Pi_O$, median $\lambda = 439$	0.944–0.962	unbounded without a public bound
Small n, public bound	200	$\Psi = I$, median $\lambda = 1{,}021$	0.946–0.962	1.000

Reading the table

- Ranges are over the 11 coefficients
- The Wald interval hits its target with or without the ridge
- The β interval is conservative: worst-case bias, and a cautious variance

For contrast

A naive Wald interval around a ridge estimate, read as an interval for β, covered only 42–81% when no eigenvalue bound was known. Naming the target matters.

Real data against central DP

Test MSE ÷ non-private at $\varepsilon = 1$, $\delta = 10^{-6}$, 10-fold cross-validation

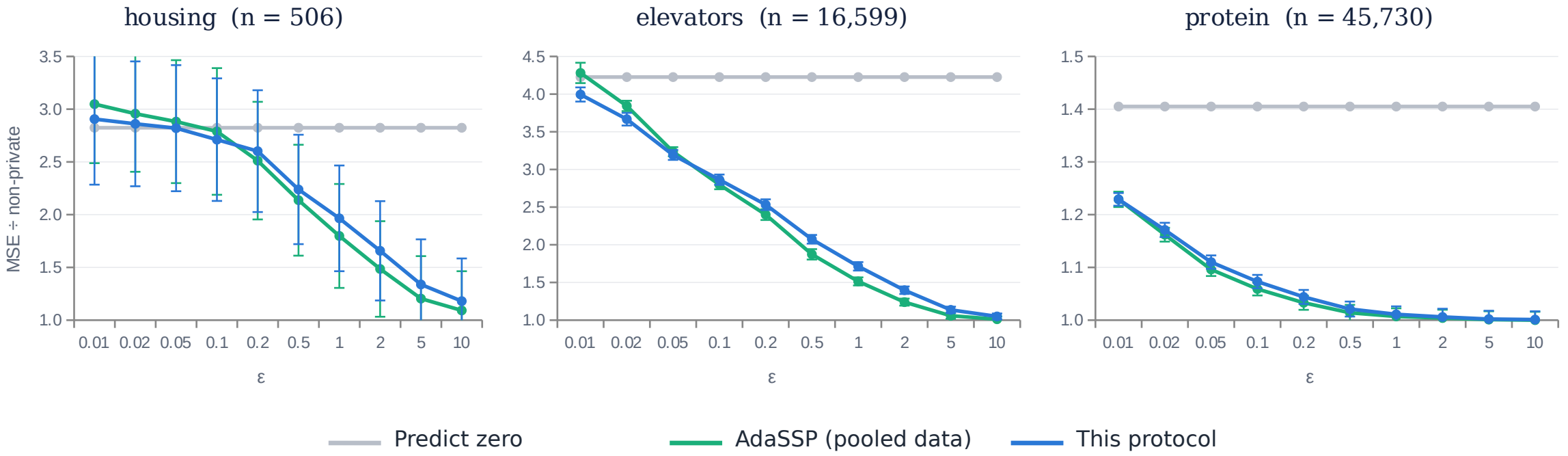
Dataset	n	d	Predict zero	AdaSSP (pooled data)	This protocol
housing	506	13	2.82×	**1.80×**	1.97×
wine	1,599	11	2.82×	**1.68×**	1.84×
airfoil	1,503	5	1.94×	**1.09×**	1.16×
concrete	1,030	8	2.87×	**1.48×**	1.69×
bike	17,379	17	3.84×	**1.03×**	1.06×
elevators	16,599	18	4.23×	**1.51×**	1.71×
pol	15,000	26	2.56×	**1.04×**	1.06×
kin40k	40,000	8	1.00×	1.00×	1.00×
protein	45,730	9	1.41×	1.01×	1.01×

1.0× = as good as non-private; lower is better; bold = better private method.

AdaSSP needs one curator with the pooled data, and protects the whole record. At nominal $\varepsilon = 1$ it is actually more private ($\varepsilon \approx 0.82$), so the comparison favours it. Here the protocol's R holds about half the features.

Across privacy levels

Test MSE ÷ non-private on three datasets of increasing size; bars: 95% CI over the 10 folds



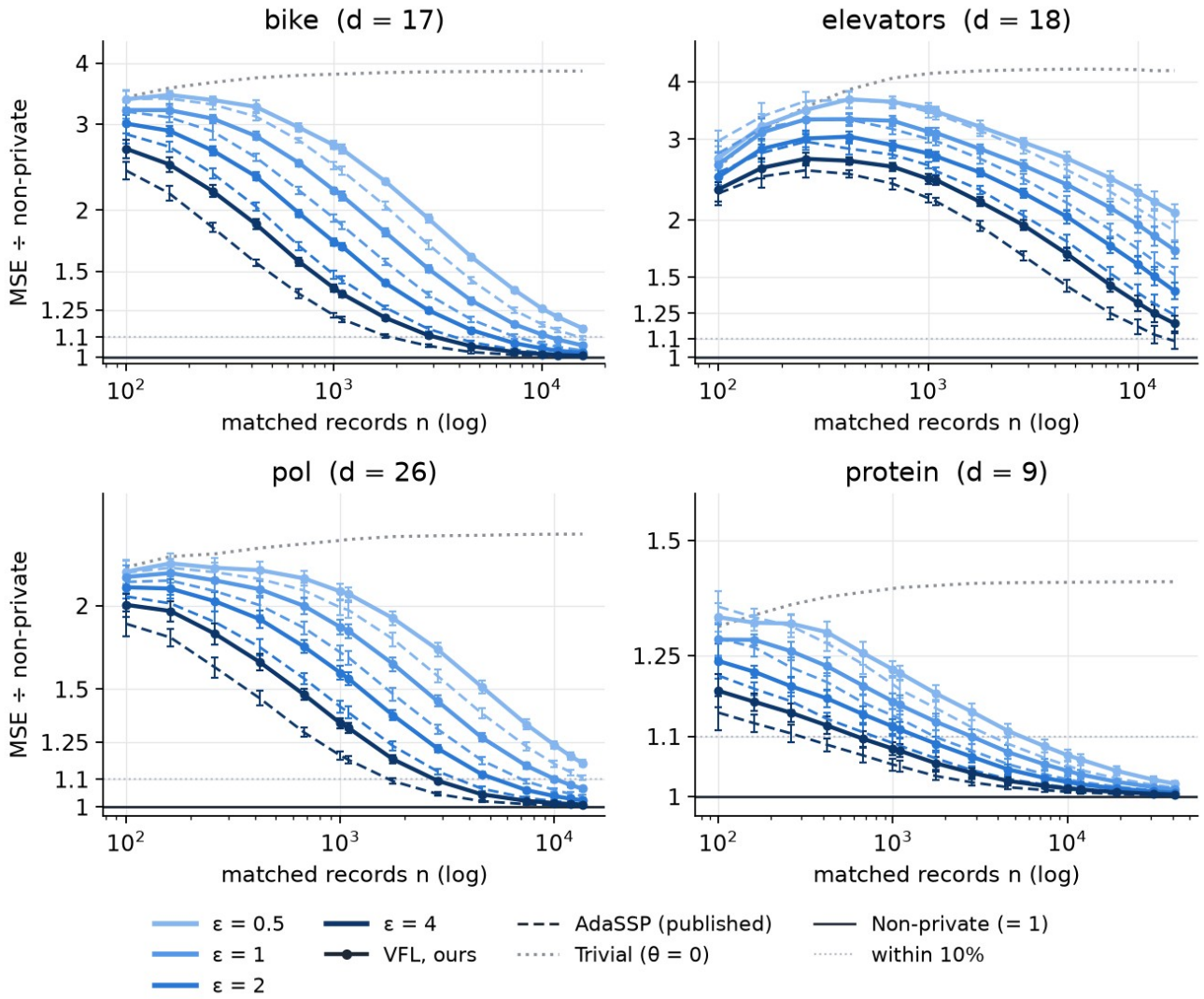
Of 29 datasets	ε = 0.1	ε = 1	ε = 10
Protocol better / worse than AdaSSP	14 / 8	6 / 19	1 / 21
Protocol worse than predicting zero	13	6	3
AdaSSP worse than predicting zero	15	7	3

Better / worse: paired over folds, 95% CI; the rest are ties.

From ε ≈ 0.5 the central method is ahead, by a median 4% at ε = 1, and the gap shrinks as n grows. At very small ε both are often no better than predicting zero.

How many matched records on real data?

Test MSE ÷ non-private at the same n; nested subsets of the training folds; bars: 95% CI over folds



Matched records to come within 10% of non-private

This protocol (AdaSSP, pooled data)

	ε = 0.5	ε = 1	ε = 2	ε = 4
bike	**>15k** (15k)	**11k** (7.5k)	**5.9k** (3.8k)	**3.1k** (1.8k)
elevators	**>14k** (>14k)	**>14k** (>14k)	**>14k** (>14k)	**>14k** (14k)
pol	**>13k** (>13k)	**10k** (7.0k)	**5.3k** (3.5k)	**2.8k** (1.7k)
protein	**5.4k** (3.7k)	**2.8k** (1.7k)	**1.4k** (784)	**619** (295)

- Doubling ε roughly halves the records needed
- About 1.5–2× the records the central method needs
- elevators never gets within 10% at the sizes available (up to 14.9k)

Limits and open items

By design

- Honest-but-curious parties only; a deviating researcher needs zero-knowledge input proofs
- R learns which of its cohort are in the register
- Exact-identifier linkage; linear models; model-based variance
- O's work grows with the padded register, so the scope Φ matters

Still to do

- Implement the full protocol in OpenFHE and measure its cost
- Rerun the experiments under the protocol's two-release rule
- Fix exact parameter sets for the larger CKKS scale
- Close the open proof items: improved circuit, adaptive composition, constant-time sampler
- A fairer real-data comparison: preprocessing each party can do on its own

Summary

1. A researcher and an agency can fit a joint regression without pooling: linkage by labeled PSI, sums under split-key encryption, DP noise added inside O's decryption share
2. O learns only public sizes; R gets (ε, δ)-DP statistics. The main guarantees are proven, for honest-but-curious parties
3. Tight public bounds and enough matched records drive accuracy: within 2× of non-private at 3k–105k records in simulation, within 10% at about 1.4k–11k on real data for ε = 1–2
4. Close to central DP on large datasets; about 1.5–2× the records on real data

To read next

- The protocol specification (algorithms 1–7)
- The proofs appendix: parts A and B
- reports/calibration: from ε to σ, λ and sample size
- reports/adassp: the real-data benchmark and sweep